Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____
Преподаватель _____С.В.Умбетов
« » _____ 2026 г.

Отчет по лабораторной работе №4
по дисциплине «Инженерно-техническая поддержка сопровождения интеллектуальных информационных систем»
«Контейнеризация компонентов ИИС и управление зависимостями (Docker)»

ЛР 09.02.08.23.004

Студент группы 1ИИС-41 М.А. Ситников
группа фи.о., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень фи.о., фамилия

Барнаул 2026

Лабораторная работа №4
Контейнеризация компонентов ИИС и управление зависимостями (Docker)

Цель работы: Целью лабораторной работы является освоение принципов контейнеризации интеллектуальных информационных систем (ИИС) с использованием Docker. В ходе выполнения работы изучается создание Dockerfile для приложений на языке Python с тяжёлыми зависимостями машинного обучения (NumPy, Scikit-learn, PyTorch/TensorFlow), методы оптимизации размера Docker-образов (multi-stage build, базовые образы slim/alpine), описание многоконтейнерных приложений с помощью docker-compose.yml (веб-интерфейс ИИС и база данных или векторное хранилище), а также организация персистентного хранения состояния системы через volumes.

Задачи:

1. Установить Docker и Docker Compose (при отсутствии). Проверить версии команд docker --version и docker compose version.
2. Создать структуру каталога iis-docker и файлы, описанные в разделах 4–8.
3. Написать multi-stage Dockerfile на базе python:3.11-slim.
4. Создать файл .dockerignore.
5. Описать docker-compose.yml с сервисами api и db (PostgreSQL) и именованными volumes.
6. Выполнить docker compose build и зафиксировать размер итогового образа (docker images).
7. Запустить систему командой docker compose up -d.
8. Проверить работоспособность API с помощью curl или браузера.
9. Убедиться, что volume model_data смонтирован и модель сохраняется между перезапусками.
10. Выполнить docker compose down, затем снова docker compose up -d и подтвердить сохранность данных.
11. (Дополнительно) Добавить в docker-compose сервис Redis или Qdrant и использовать его для кэширования предсказаний.

Задание принято:
Ситников М.А.



Подпись

Ход работы

1. Установить Docker и Docker Compose (при отсутствии). Проверить версии команд docker --version и docker compose version.

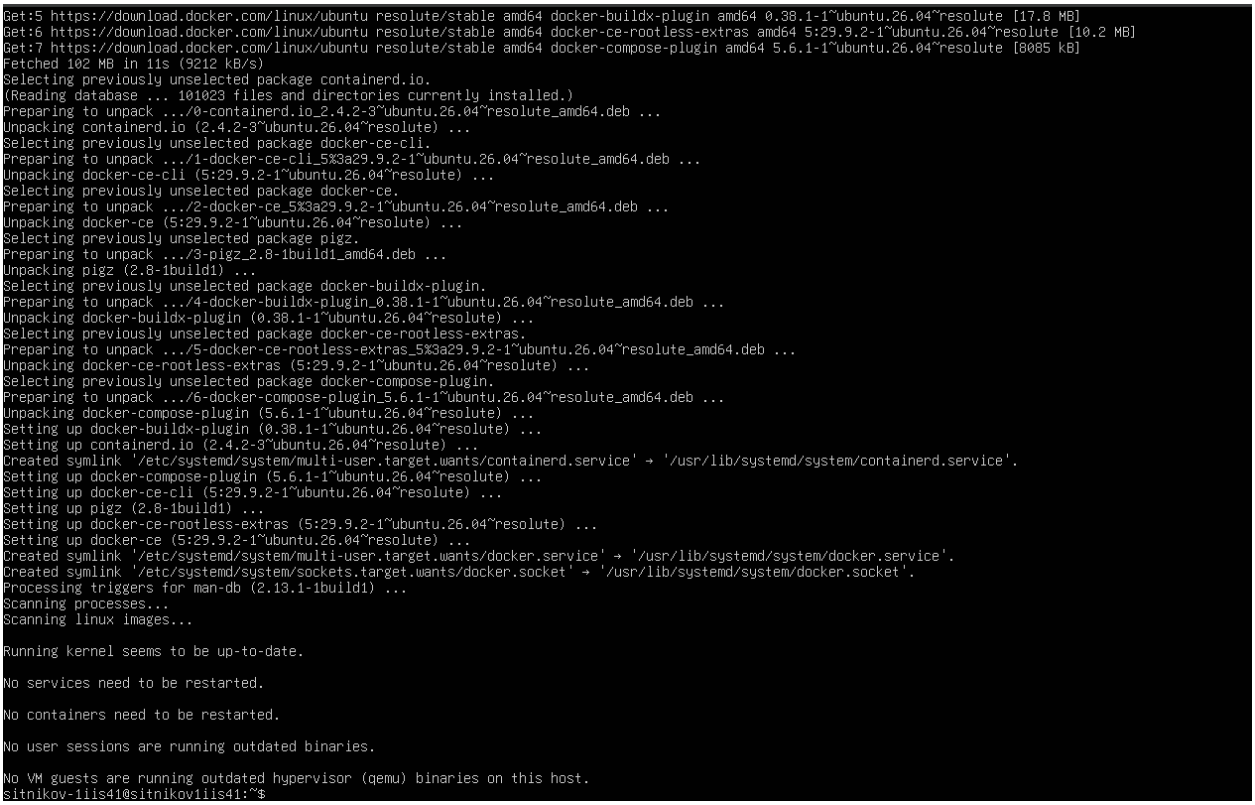


Рис.1 - Подготовка системы и ключа Docker, подключение репозитория и установка Docker

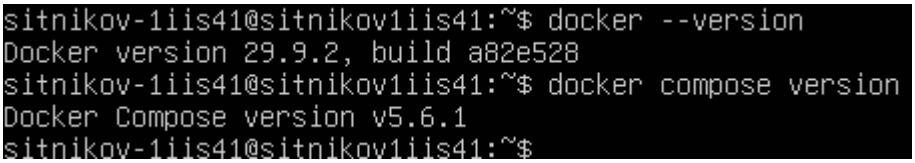


Рис.2 – Проверка веерии с помощью команд docker --version и docker compose version

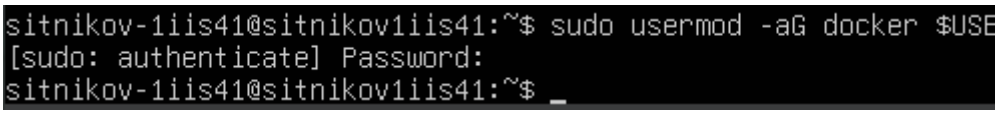


Рис.3 – Добавление пользователя в группу docker командой sudo usermod –aG docker $USER

2. Создать структуру каталога iis-docker и файлы, описанные в разделах 4–8.



Рис.4 – Создана структура каталога iis-docker. Созданы файлы из разделов 4-8

```
  GNU nano 8.7.1
fastapi==0.115.0
uvicorn[standard]==0.30.6

scikit-learn==1.5.2
numpy==2.1.1
pandas==2.2.3

psycopg2-binary==2.9.9
pydantic==2.9.2
```

Рис.5 – Заполнение app/requirements.txt для фиксирования версии пакетов необходимых приложению

```python
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassfier
import joblib
import os

MODEL_PATH='/data/model.joblib'

def train_and_save():
        X,y=load_iris(return_X_y=True)
        model=RandomForestClassifier(n_estimators=100, random_state=42)
        model.fit(X,y)
        os.makedirs(os.path.dirname(MODEL_PATH), exist_ok=True)
        joblib.dump(model, MODEL_PATH)
        return model

def load_model():
        if os.path.exists(MODEL_PATH):
                return joblib.load(MODEL_PATH)
        return train_and_save()
```

Рис.6 – Заполнение app/model.py для обучение модели RandomForest на наборе Iris и сохранения/загрузки сериализованной модели из каталога /data

```python
from fastapi import FastAPI
from pydantic import BaseModel
from model import load_model

app = FastAPI(title="IIS Iris Classifier")

model = load_model()

class IrisFeatures(BaseModel):
        sepal_length: float
        sepal_width: float
        petal_length: float
        petal_width: float

@app.get("/")
def root():
        return {"status": "ok", "service": "IIS Iris Classifier"}

@app.post("/predict")
def predict(features: IrisFeatures):
        data = [[
                features.sepal_length,
                features.sepal_width,
                features.petal_length,
                features.petal_width,
        ]]
        pred = model.predict(data)[0]
        return {"predicted_class": int(pred)}
```

Рис.7 – Заполнение app/main для эндпоинт проверки состояния и эндпоинт предсказания класса по четырём признакам цветка ирис

3. Написать multi-stage Dockerfile на базе python:3.11-slim.

```
FROM python:3.11-slim AS builder

WORKDIR /build

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    gcc \
    && rm -rf /var/lib/apt/lists/*

COPY app/requirements.txt .

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim AS runtime

WORKDIR /app

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY app/ .

RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Рис.8 – Написан multi-stage Dockerfile на базе python:3.11-slim с помощью nano.

4. Создать файл .dockerignore.

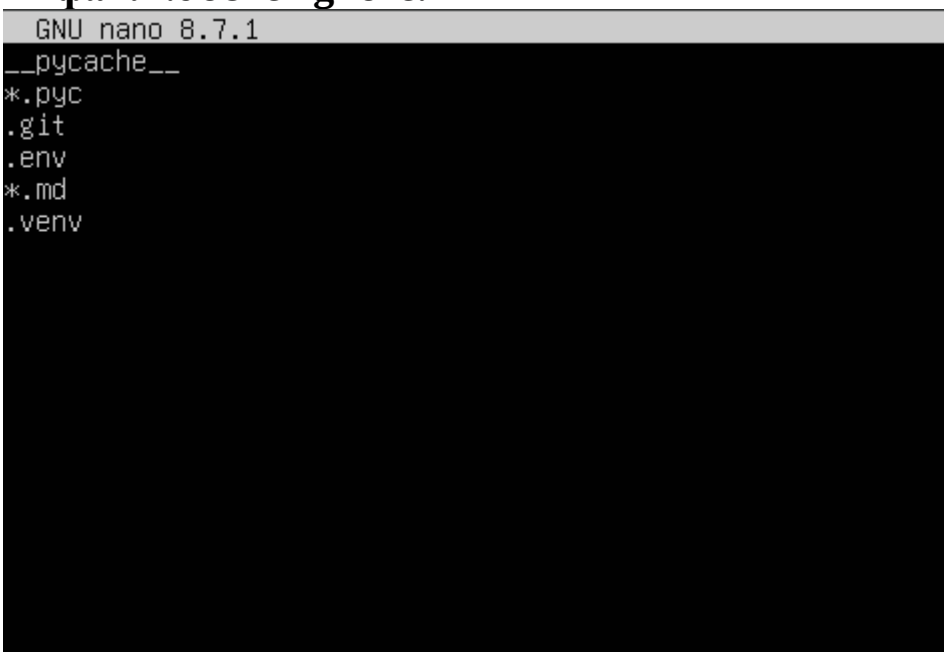


Рис.9 – Файл .dockerignore создан и заполнен с помощью nano

5. Описать docker-compose.yml с сервисами api и db (PostgreSQL) и именованными volumes.



Рис.10 – docker-compose.yml описан с сервисами api и db (PostgreSQL) и именованными volumes.

6. Выполнить docker compose build и зафиксировать размер итогового образа (docker images).



Рис.11 – docker compose build выполнен



Рис.12 – Размер итогового образа зафиксирован с помощью docker images.

Результат: Итоговый размер образа: 766 Мб

7. Запустить систему командой docker compose up -d.



Рис.13 – Система запущена командой docker compose up –d

8. Проверить работоспособность API с помощью curl или браузера.

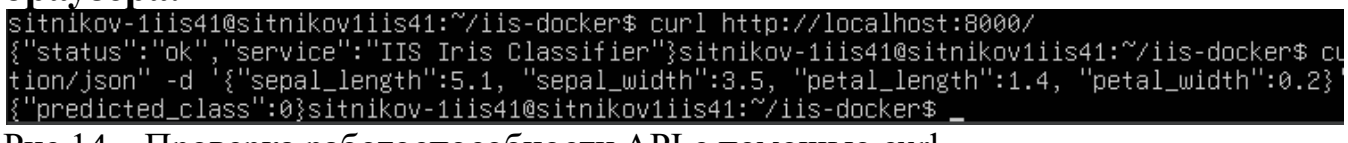


Рис.14 – Проверка работоспособности API с помощью curl.

Результат: API работает правильно. Модель принимает признаки цветка ирис и возвращает правильно предсказанный класс (0).

9. Убедиться, что volume model_data смонтирован и модель сохраняется между перезапусками.

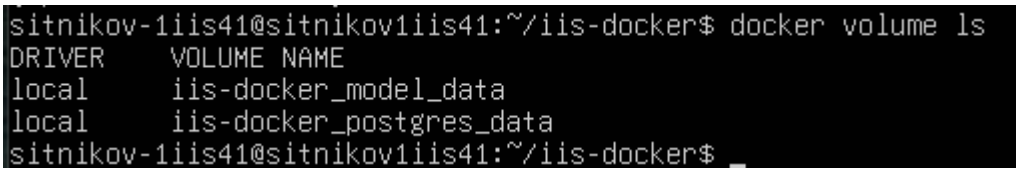


Рис.15 – Список volume проекта выведенный с помощью docker volume ls



Рис.16 – Подробности о volume с моделью выведенные с помощью docker volume inspect iis-docker_model_data



Рис.17 – Содержимое /data внутри контейнера API выведенное командой docker compose exec api ls -la /data

10. Выполнить docker compose down, затем снова docker compose up -d и подтвердить сохранность данных.

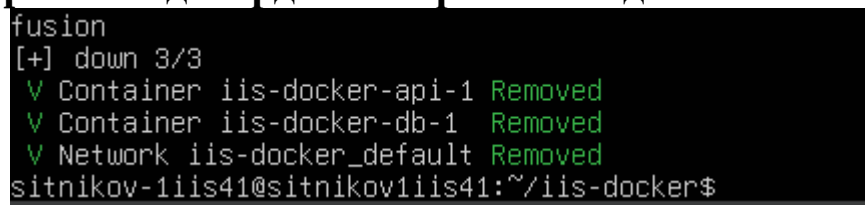


Рис.18 – Остановка и удаление контейнеров с помощью docker compose down



Рис.19 – Повторный запуск с помощью docker compose up –d

```
fusion
total 192
drwxr-xr-x 2 root root   4096 Nov  1 16:18 .
drwxr-xr-x 1 root root   4096 Nov  2 03:02 ..
-rw-r--r-- 1 root root 186753 Nov  1 16:18 model.joblib
sitnikov-1iis41@sitnikov1iis41:~/iis-docker$
```

Рис.20 – Проверка сохранности данных с помощью docker compose exec api ls -la /data

Результат: После остановки и повторного запуска все данные сохранились

11. (Дополнительно) Добавить в docker-compose сервис Redis или Qdrant и использовать его для кэширования предсказаний.

```yaml
version: "3.9"

services:
  api:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - model_data:/data
      - ./logs:/app/logs
    environment:
      - DATABASE_URL=postgresql://iis:iispass@db:5432/iisdb
      - REDIS_URL=redis://redis:6379/0
    depends_on:
      - db
      - redis
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: iis
      POSTGRES_PASSWORD: iispass
      POSTGRES_DB: iisdb
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    restart: unless-stopped

volumes:
  model_data:
  postgres_data:
```

Рис.21 – Сервис Redis добавлен и подключен к api

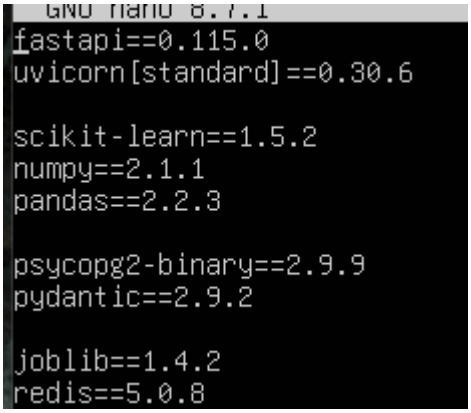


Рис.22 – Добавление Redis в app/requirements.txt

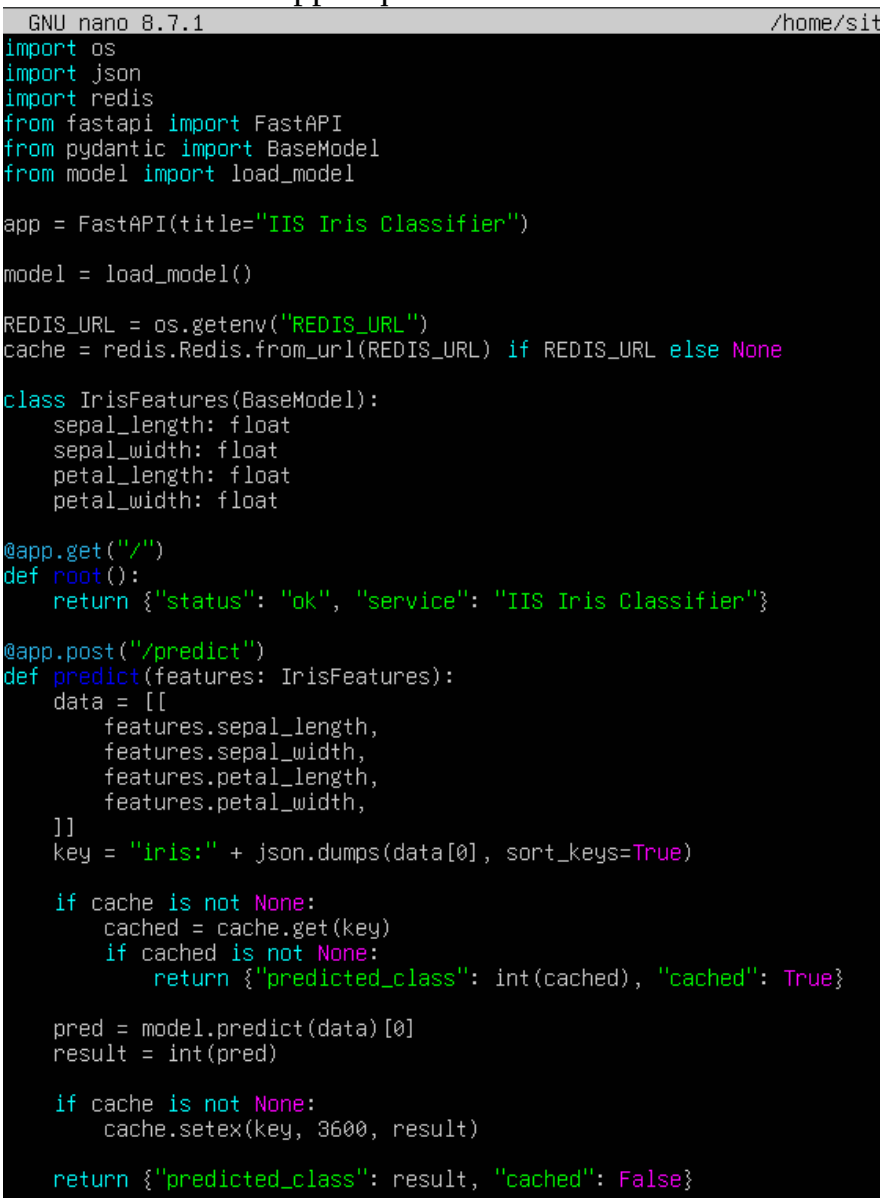


Рис.23 – Добавление кеширования в app/main.py

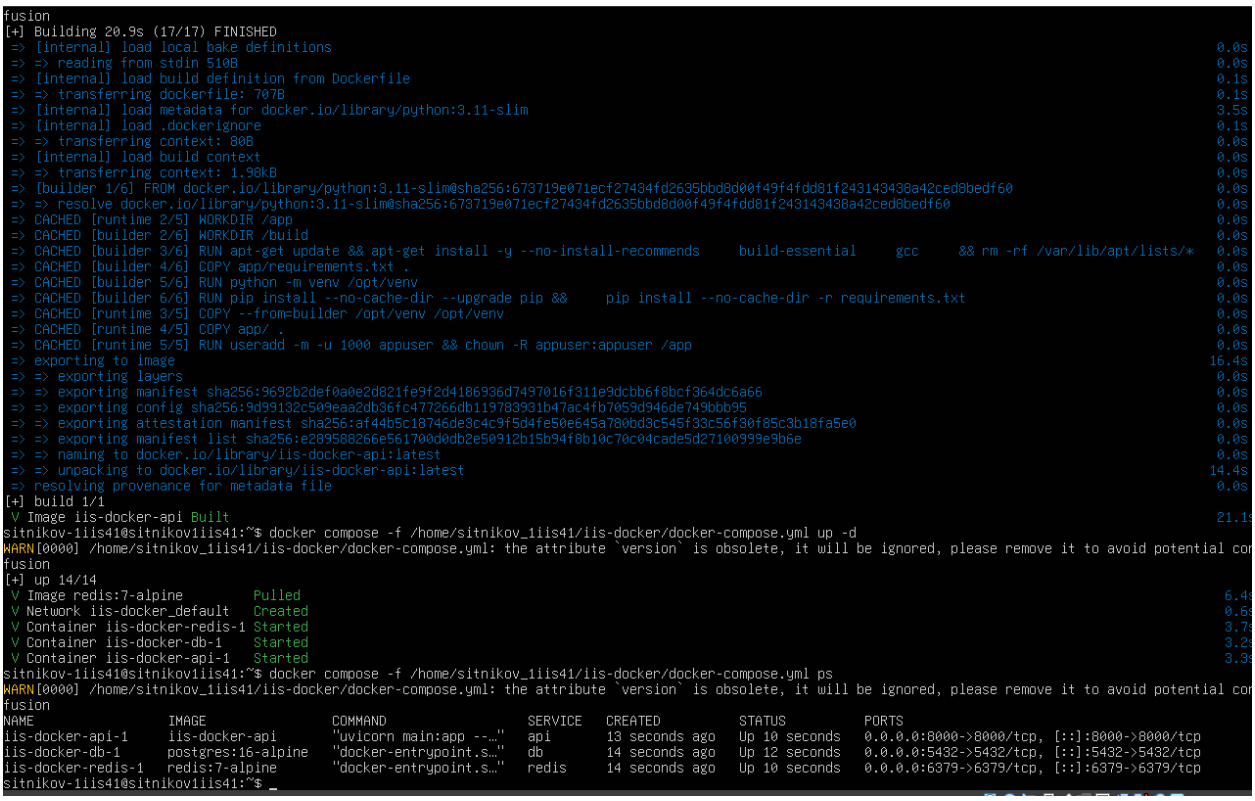


Рис.24 – Пересборка и перезапуск docker с помощью docker compose build и docker compose up -d. Проверка состояния сервисов с помощью docker compose ps

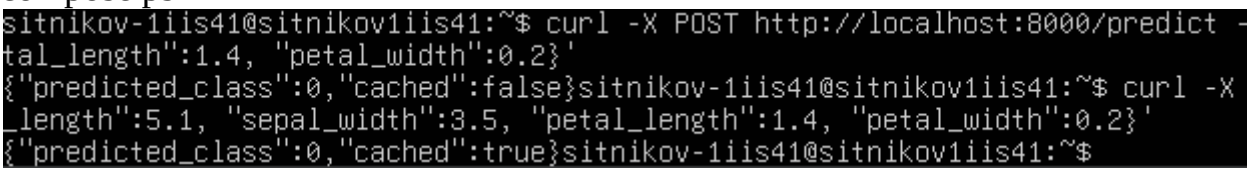


Рис.25 – Проверка работоспособности API с помощью curl.

Результат: Первый запрос возвращает "cached": false (модель сделала предсказание), а при повторном запросе возвращается "cached": true (ответ взят из кэша).

Ссылка на GitHub:
https://github.com/Enot2130/Sitnikov_1iis41/tree/main/Инженерно-техническая%20поддержка%20сопровождения%20интеллектуальных%20информационных%20систем/Контейнеризация%20компонентов%20ИИС%20и%20управление%20зависимостями%20(Docker)

Контрольные вопросы

1. Что такое Docker-образ и чем он отличается от контейнера?
Образ — неизменяемый шаблон; контейнер — запущенный экземпляр образа.

2. Для чего используется multi-stage build и какую проблему он решает при контейнеризации систем машинного обучения?
Multi-stage build — техника сборки, при которой в одном Dockerfile используется несколько стадий. На первой стадии устанавливаются

зависимости и собирается приложение; на финальной стадии в лёгкий образ копируются только необходимые артефакты.

3. Почему рекомендуется использовать образы python:*-slim или alpine вместо полноразмерных?

Лёгкие базовые образы вместо полноразмерных уменьшают размер итогового образа.

4. Что такое слой (layer) в Docker и почему важен порядок инструкций в Dockerfile?

Слой — результат одной инструкции Dockerfile. Порядок влияет на кэширование: редко меняющееся (зависимости) ставят раньше часто меняющегося (код).

5. Для чего предназначен файл .dockerignore?

Исключает лишние файлы из контекста сборки (__pycache__, .git, .env, .venv и т.п.).

6. В чём различие между bind-mount и named volume?

Named volume (например model_data) управляется Docker-ом и сохраняет данные между перезапусками; bind-mount (например ./logs) монтирует конкретный каталог хоста.

7. Как обеспечить сохранение состояния модели и базы данных при пересоздании контейнеров?

С помощью Named volumes в docker-compose.yml (model_data:/data и postgres_data:/var/lib/postgresql/data). docker compose down их не удаляет.

8. Что выполняет инструкция COPY --from=builder?

Копирует готовые виртуальные окружения из стадии builder в финальный образ без компиляторов.

9. Зачем запускать процесс приложения от имени непривилегированного пользователя?

Для безопасности: запуск от непривилегированного пользователя снижает риски, если приложение будет взломано.

10. Как просмотреть логи отдельного сервиса в docker-compose?

docker compose logs -f api.

11. Какими способами можно уменьшить размер образа при использовании PyTorch или TensorFlow?

slim-база, multi-stage, --no-cache-dir, .dockerignore, вынос модели в volume.

12. Что означает параметр depends_on и гарантирует ли он готовность зависимого сервиса?

Задаёт порядок запуска, но не ждёт готовности сервиса.

13. Как определить, какие volumes использует проект?

Выполнить команды: docker volume ls, docker volume inspect iis-docker_model_data, плюс секция volumes: в compose-файле.

14. В чём преимущество контейнеризации ИИС по сравнению с виртуальными машинами?

Одинаковая работа системы на машине разработчика, сервере и в облаке, изоляция со всеми зависимостями.

15. Какие риски возникают при хранении паролей и секретов

непосредственно в docker-compose.yml?
Пароли хранятся открыто и могут утечь при публикации файла Для защиты секретов .env исключён в .dockerignore.

Вывод

В ходе лабораторной работы я освоил контейнеризацию ИИС с помощью Docke: установил Docker и Docker Compose, создал FastAPI-приложение с моделью scikit-learn, написал multi-stage Dockerfile на базе python:3.11-slim с .dockerignore, описал docker-compose.yml с сервисами api и db (PostgreSQL) и именованными volumes. Я собрал и запустил систему, проверил API через curl, убедился, что модель сохраняется между перезапусками благодаря volume model_data, и дополнительно добавил Redis для кэширования предсказаний. В результате я убедился, что контейнеризация обеспечивает воспроизводимость работы ИИС в разных средах, а multi-stage сборка, лёгкие образы и .dockerignore уменьшают размер образа, тогда как named volumes сохраняют состояние вне жизненного цикла контейнеров.